

tempfile — Generate temporary files and directories

Source code: [Lib/tempfile.py](#)

This module creates temporary files and directories. It works on all supported platforms.

[TemporaryFile](#), [NamedTemporaryFile](#), [TemporaryDirectory](#), and [SpooledTemporaryFile](#) are high-level interfaces which provide automatic cleanup and can be used as [context managers](#). [mkstemp\(\)](#) and [mkdtemp\(\)](#) are lower-level functions which require manual cleanup.

All the user-callable functions and constructors take additional arguments which allow direct control over the location and name of temporary files and directories. Files names used by this module include a string of random characters which allows those files to be securely created in shared temporary directories. To maintain backward compatibility, the argument order is somewhat odd; it is recommended to use keyword arguments for clarity.

The module defines the following user-callable items:

tempfile.TemporaryFile(*mode='w+b', buffering=-1, encoding=None, newline=None, suffix=None, prefix=None, dir=None, *, errors=None*)

Return a [file-like object](#) that can be used as a temporary storage area. The file is created securely, using the same rules as [mkstemp\(\)](#). It will be destroyed as soon as it is closed (including an implicit close when the object is garbage collected). Under Unix, the directory entry for the file is either not created at all or is removed immediately after the file is created. Other platforms do not support this; your code should not rely on a temporary file created using this function having or not having a visible name in the file system.

The resulting object can be used as a [context manager](#) (see [Examples](#)). On completion of the context or destruction of the file object the temporary file will be removed from the filesystem.

The *mode* parameter defaults to 'w+b' so that the file created can be read and written without being closed. Binary mode is used so that it behaves consistently on all platforms without regard for the data that is stored. *buffering*, *encoding*, *errors* and *newline* are interpreted as for [open\(\)](#).

The *dir*, *prefix* and *suffix* parameters have the same meaning and defaults as with [mkstemp\(\)](#).

The returned object is a true file object on POSIX platforms. On other platforms, it is a file-like object whose *file* attribute is the underlying true file object.

The [os.O_TMPFILE](#) flag is used if it is available and works (Linux-specific, requires Linux kernel 3.11 or later).

On platforms that are neither Posix nor Cygwin, TemporaryFile is an alias for NamedTemporaryFile.

Raises an [auditing event](#) `tempfile.mkstemp` with argument `fullpath`.

Changed in version 3.5: The [os.O_TMPFILE](#) flag is now used if available.

Changed in version 3.8: Added `errors` parameter.

```
tempfile.NamedTemporaryFile(mode='w+b', buffering=-1, encoding=None,  
newline=None, suffix=None, prefix=None, dir=None, delete=True, *, errors=None,  
delete_on_close=True)
```

This function operates exactly as [TemporaryFile\(\)](#) does, except the following differences:

- This function returns a file that is guaranteed to have a visible name in the file system.
- To manage the named file, it extends the parameters of [TemporaryFile\(\)](#) with `delete` and `delete_on_close` parameters that determine whether and how the named file should be automatically deleted.

The returned object is always a [file-like object](#) whose `file` attribute is the underlying true file object. This file-like object can be used in a [with](#) statement, just like a normal file. The name of the temporary file can be retrieved from the `name` attribute of the returned file-like object. On Unix, unlike with the [TemporaryFile\(\)](#), the directory entry does not get unlinked immediately after the file creation.

If `delete` is true (the default) and `delete_on_close` is true (the default), the file is deleted as soon as it is closed. If `delete` is true and `delete_on_close` is false, the file is deleted on context manager exit only, or else when the [file-like object](#) is finalized. Deletion is not always guaranteed in this case (see [object.__del__\(\)](#)). If `delete` is false, the value of `delete_on_close` is ignored.

Therefore to use the name of the temporary file to reopen the file after closing it, either make sure not to delete the file upon closure (set the `delete` parameter to be false) or, in case the temporary file is created in a [with](#) statement, set the `delete_on_close` parameter to be false. The latter approach is recommended as it provides assistance in automatic cleaning of the temporary file upon the context manager exit.

Opening the temporary file again by its name while it is still open works as follows:

- On POSIX the file can always be opened again.
- On Windows, make sure that at least one of the following conditions are fulfilled:
 - `delete` is false
 - additional open shares delete access (e.g. by calling [os.open\(\)](#) with the flag `O_TEMPORARY`)
 - `delete` is true but `delete_on_close` is false. Note, that in this case the additional opens that do not share delete access (e.g. created via builtin [open\(\)](#)) must be closed before exiting the context manager, else the [os.unlink\(\)](#) call on context manager exit will fail with a [PermissionError](#).

On Windows, if `delete_on_close` is false, and the file is created in a directory for which the user lacks delete access, then the [os.unlink\(\)](#) call on exit of the context manager will fail with a [PermissionError](#). This cannot happen when `delete_on_close` is true because delete access is requested by the open, which fails immediately if the requested access is not granted.

On POSIX (only), a process that is terminated abruptly with SIGKILL cannot automatically delete any NamedTemporaryFiles it created.

Raises an [auditing event](#) `tempfile.mkstemp` with argument `fullpath`.

Changed in version 3.8: Added `errors` parameter.

Changed in version 3.12: Added `delete_on_close` parameter.

```
class tempfile.SpoiledTemporaryFile(max_size=0, mode='w+b', buffering=-1,
encoding=None, newline=None, suffix=None, prefix=None, dir=None, *,
errors=None)
```

This class operates exactly as [TemporaryFile\(\)](#) does, except that data is spooled in memory until the file size exceeds `max_size`, or until the file's [fileno\(\)](#) method is called, at which point the contents are written to disk and operation proceeds as with `TemporaryFile()`.

rollover()

The resulting file has one additional method, `rollover()`, which causes the file to roll over to an on-disk file regardless of its size.

The returned object is a file-like object whose `_file` attribute is either an [io.BytesIO](#) or [io.TextIOWrapper](#) object (depending on whether binary or text `mode` was specified) or a true file object, depending on whether [rollover\(\)](#) has been called. This file-like object can be used in a [with](#) statement, just like a normal file.

Changed in version 3.3: the `truncate` method now accepts a `size` argument.

Changed in version 3.8: Added `errors` parameter.

Changed in version 3.11: Fully implements the [io.BufferedIOBase](#) and [io.TextIOBase](#) abstract base classes (depending on whether binary or text `mode` was specified).

```
class tempfile.TemporaryDirectory(suffix=None, prefix=None, dir=None,
ignore_cleanup_errors=False, *, delete=True)
```

This class securely creates a temporary directory using the same rules as [mkdtemp\(\)](#). The resulting object can be used as a [context manager](#) (see [Examples](#)). On completion of the context or destruction of the temporary directory object, the newly created temporary directory and all its contents are removed from the filesystem.

name

The directory name can be retrieved from the `name` attribute of the returned object. When the returned object is used as a [context manager](#), the name will be assigned to the target of the `as` clause in the [with](#) statement, if there is one.

cleanup()

The directory can be explicitly cleaned up by calling the `cleanup()` method. If `ignore_cleanup_errors` is true, any unhandled exceptions during explicit or implicit cleanup (such as a [PermissionError](#) removing open files on Windows) will be ignored, and the remaining removable items deleted on a "best-effort" basis. Otherwise, errors will be raised in whatever context cleanup occurs (the `cleanup()` call, exiting the context manager, when the object is garbage-collected or during interpreter shutdown).

The `delete` parameter can be used to disable cleanup of the directory tree upon exiting the context. While it may seem unusual for a context manager to disable the action taken when exiting the

context, it can be useful during debugging or when you need your cleanup behavior to be conditional based on other logic.

Raises an [auditing event](#) `tempfile.mkdtemp` with argument `fullpath`.

Added in version 3.2.

Changed in version 3.10: Added `ignore_cleanup_errors` parameter.

Changed in version 3.12: Added the `delete` parameter.

`tempfile.mkstemp(suffix=None, prefix=None, dir=None, text=False)`

Creates a temporary file in the most secure manner possible. There are no race conditions in the file's creation, assuming that the platform properly implements the [os.O_EXCL](#) flag for [os.open\(\)](#). The file is readable and writable only by the creating user ID. If the platform uses permission bits to indicate whether a file is executable, the file is executable by no one.

The file descriptor is [not inherited by child processes](#).

Unlike [TemporaryFile\(\)](#), the user of `mkstemp()` is responsible for deleting the temporary file when done with it.

If `suffix` is not `None`, the file name will end with that suffix, otherwise there will be no suffix. `mkstemp()` does not put a dot between the file name and the suffix; if you need one, put it at the beginning of `suffix`.

If `prefix` is not `None`, the file name will begin with that prefix; otherwise, a default prefix is used. The default is the return value of [gettempprefix\(\)](#) or [gettempprefixb\(\)](#), as appropriate.

If `dir` is not `None`, the file will be created in that directory; otherwise, a default directory is used. The default directory is chosen from a platform-dependent list, but the user of the application can control the directory location by setting the `TMPDIR`, `TEMP` or `TMP` environment variables. There is thus no guarantee that the generated filename will have any nice properties, such as not requiring quoting when passed to external commands via `os.popen()`.

If any of `suffix`, `prefix`, and `dir` are not `None`, they must be the same type. If they are bytes, the returned name will be bytes instead of `str`. If you want to force a bytes return value with otherwise default behavior, pass `suffix=b''`.

If `text` is specified and `true`, the file is opened in text mode. Otherwise, (the default) the file is opened in binary mode.

`mkstemp()` returns a tuple containing an OS-level handle to an open file (as would be returned by [os.open\(\)](#)) and the absolute pathname of that file, in that order.

Raises an [auditing event](#) `tempfile.mkstemp` with argument `fullpath`.

Changed in version 3.5: `suffix`, `prefix`, and `dir` may now be supplied in bytes in order to obtain a bytes return value. Prior to this, only `str` was allowed. `suffix` and `prefix` now accept and default to `None` to cause an appropriate default value to be used.

Changed in version 3.6: The `dir` parameter now accepts a [path-like object](#).

`tempfile.mkdtemp(suffix=None, prefix=None, dir=None)`

Creates a temporary directory in the most secure manner possible. There are no race conditions in the directory's creation. The directory is readable, writable, and searchable only by the creating user ID.

The user of `mkdtemp()` is responsible for deleting the temporary directory and its contents when done with it.

The `prefix`, `suffix`, and `dir` arguments are the same as for [mkstemp\(\)](#).

`mkdtemp()` returns the absolute pathname of the new directory.

Raises an [auditing event](#) `tempfile.mkdtemp` with argument `fullpath`.

Changed in version 3.5: `suffix`, `prefix`, and `dir` may now be supplied in bytes in order to obtain a bytes return value. Prior to this, only str was allowed. `suffix` and `prefix` now accept and default to `None` to cause an appropriate default value to be used.

Changed in version 3.6: The `dir` parameter now accepts a [path-like object](#).

Changed in version 3.12: `mkdtemp()` now always returns an absolute path, even if `dir` is relative.

`tempfile.gettempdir()`

Return the name of the directory used for temporary files. This defines the default value for the `dir` argument to all functions in this module.

Python searches a standard list of directories to find one which the calling user can create files in. The list is:

1. The directory named by the `TMPDIR` environment variable.
2. The directory named by the `TEMP` environment variable.
3. The directory named by the `TMP` environment variable.
4. A platform-specific location:
 - On Windows, the directories `C:\TEMP`, `C:\TMP`, `\TEMP`, and `\TMP`, in that order.
 - On all other platforms, the directories `/tmp`, `/var/tmp`, and `/usr/tmp`, in that order.
5. As a last resort, the current working directory.

The result of this search is cached, see the description of [tempdir](#) below.

Changed in version 3.10: Always returns a str. Previously it would return any [tempdir](#) value regardless of type so long as it was not `None`.

`tempfile.gettempdirb()`

Same as [gettempdir\(\)](#) but the return value is in bytes.

Added in version 3.5.

`tempfile.gettempprefix()`

Return the filename prefix used to create temporary files. This does not contain the directory component.

`tempfile.gettempprefixb()`

Same as [gettempprefix\(\)](#) but the return value is in bytes.

Added in version 3.5.

The module uses a global variable to store the name of the directory used for temporary files returned by [gettempdir\(\)](#). It can be set directly to override the selection process, but this is discouraged. All functions in this module take a *dir* argument which can be used to specify the directory. This is the recommended approach that does not surprise other unsuspecting code by changing global API behavior.

`tempfile.tempdir`

When set to a value other than `None`, this variable defines the default value for the *dir* argument to the functions defined in this module, including its type, bytes or str. It cannot be a [path-like object](#).

If `tempdir` is `None` (the default) at any call to any of the above functions except [gettempprefix\(\)](#) it is initialized following the algorithm described in [gettempdir\(\)](#).

Note: Beware that if you set `tempdir` to a bytes value, there is a nasty side effect: The global default return type of [mkstemp\(\)](#) and [mkdtemp\(\)](#) changes to bytes when no explicit prefix, suffix, or *dir* arguments of type str are supplied. Please do not write code expecting or depending on this. This awkward behavior is maintained for compatibility with the historical implementation.

Examples

Here are some examples of typical usage of the `tempfile` module:

```
>>> import tempfile

# create a temporary file and write some data to it
>>> fp = tempfile.TemporaryFile()
>>> fp.write(b'Hello world!')
# read data from file
>>> fp.seek(0)
>>> fp.read()
b'Hello world!'
# close the file, it will be removed
>>> fp.close()

# create a temporary file using a context manager
>>> with tempfile.TemporaryFile() as fp:
...     fp.write(b'Hello world!')
...     fp.seek(0)
...     fp.read()
b'Hello world!'
>>>
# file is now closed and removed
```

```

# create a temporary file using a context manager
# close the file, use the name to open the file again
>>> with tempfile.NamedTemporaryFile(delete_on_close=False) as fp:
...     fp.write(b'Hello world!')
...     fp.close()
...     # the file is closed, but not removed
...     # open the file again by using its name
...     with open(fp.name, mode='rb') as f:
...         f.read()
b'Hello world!'
>>>
# file is now removed

# create a temporary directory using the context manager
>>> with tempfile.TemporaryDirectory() as tmpdirname:
...     print('created temporary directory', tmpdirname)
>>>
# directory and contents have been removed

```

Deprecated functions and variables

A historical way to create temporary files was to first generate a file name with the [mktemp\(\)](#) function and then create a file using this name. Unfortunately this is not secure, because a different process may create a file with this name in the time between the call to `mktemp()` and the subsequent attempt to create the file by the first process. The solution is to combine the two steps and create the file immediately. This approach is used by [mkstemp\(\)](#) and the other functions described above.

`tempfile.mktemp(suffix='', prefix='tmp', dir=None)`

Deprecated since version 2.3: Use [mkstemp\(\)](#) instead.

Return an absolute pathname of a file that did not exist at the time the call is made. The *prefix*, *suffix*, and *dir* arguments are similar to those of [mkstemp\(\)](#), except that bytes file names, `suffix=None` and `prefix=None` are not supported.

Warning: Use of this function may introduce a security hole in your program. By the time you get around to doing anything with the file name it returns, someone else may have beaten you to the punch. `mktemp()` usage can be replaced easily with [NamedTemporaryFile\(\)](#), passing it the `delete=False` parameter:

```

>>> f = NamedTemporaryFile(delete=False)
>>> f.name
'/tmp/tmptjujtt'
>>> f.write(b"Hello World!\n")
13
>>> f.close()
>>> os.unlink(f.name)
>>> os.path.exists(f.name)
False

```